

Trabajo Práctico: Comunicación Inter-Core mediante Queues en FreeRTOS

Materia: Sistemas de Tiempo Real – Unidad 4

Tecnología: ESP32, FreeRTOS, Sincronización asíncrona, Colas (Queues).

1. Introducción

En sistemas multitarea, el intercambio de información entre tareas de distintas prioridades o que residen en distintos núcleos no debe realizarse mediante variables globales simples, ya que esto genera **condiciones de carrera** y corrupción de datos.

FreeRTOS provee las **Queues (Colas)**: mecanismos de comunicación seguros que funcionan por copia de datos, permitiendo que una tarea "Productora" envíe información a una "Consumidora" de forma ordenada y determinista.

2. Objetivo

Implementar un sistema de telemetría de doble núcleo donde los datos capturados por un sensor simulado se procesen en un núcleo distinto al de adquisición, garantizando el desacoplamiento de las tareas.

3. Requerimientos del Sistema

A. Tarea Productora (Productor - Core 1)

- **Prioridad:** Media (2).
- **Frecuencia:** Ejecución cada 500ms.
- **Función:** Simular la lectura de un sensor analógico (generar un número aleatorio entre 0 y 100).
- **Acción:** Enviar el valor a la cola. Si la cola está llena, la tarea debe esperar un máximo de 10ms antes de descartar el dato.

B. Tarea de Procesamiento (Consumidor - Core 0)

- **Prioridad:** Baja (1).
- **Función:** Permanecer en estado **Blocked** hasta que haya datos disponibles en la cola.
- **Acción:** Al recibir un dato, debe imprimir en la terminal:
 1. El valor recibido.
 2. El núcleo en el que se está ejecutando (usar `xPortGetCoreID()`).
 3. Un mensaje de "Procesando..." para simular carga de trabajo.

11/05/2026

C. Estructura de la Cola

- **Capacidad:** 10 elementos.
- **Tipo de dato:** Entero (`int`).

4. Requerimientos Técnicos Obligatorios

1. **Instanciación:** La cola debe ser creada antes del lanzamiento de las tareas mediante `xQueueCreate`.
2. **Seguridad:** Utilizar `xQueueSend` y `xQueueReceive` verificando siempre los valores de retorno (`pdTRUE` / `pdFALSE`).
3. **Aislamiento:** Las tareas deben estar ancladas a núcleos diferentes usando `xTaskCreatePinnedToCore`.